

Matrix Multiplication Program

CFD Assignment 3 — Reading Two Matrices from File, Multiplying, and Writing the Result

Name: Anju Farswan Roll No.: ME24BT016

1. Objective

Write a C/C++ program that reads two matrices A and B from files "A.dat" and "B.dat", checks whether they are compatible for multiplication (columns of A = rows of B), computes the product $C = A \times B$ if possible, prints C to the screen, and writes C to "C.dat".

2. Algorithm (Pseudocode)

```
FUNCTION readMatrixFromFile(filename):
    OPEN filename; IF fails -> print error, EXIT
    FOR each line READ from file:
        REPLACE commas with spaces
        EXTRACT all numbers on the line -> rowValues
        IF rowValues is empty -> SKIP line (blank line)
        IF this is the first data row -> SET expectedCols = size(rowValues)
        ELSE IF size(rowValues) != expectedCols -> print error, EXIT
        APPEND rowValues to matrix
    rows = number of rows collected; cols = expectedCols
    RETURN matrix

FUNCTION multiplyMatrices(A, B, r1, c1, r2, c2):
    IF c1 != r2 -> print "Matrix multiplication is not possible.", EXIT
    CREATE C as (r1 x c2) matrix of zeros
    FOR i = 0 TO r1-1:
        FOR j = 0 TO c2-1:
            FOR k = 0 TO c1-1:
                C[i][j] += A[i][k] * B[k][j]
    RETURN C

FUNCTION displayMatrix / writeMatrixToFile:
    PRINT/WRITE each row of the matrix, fixed to OUTPUT_PRECISION decimals

MAIN:
    A <- readMatrixFromFile("A.dat")
    B <- readMatrixFromFile("B.dat")
    C <- multiplyMatrices(A, B, r1, c1, r2, c2)
    DISPLAY C
    writeMatrixToFile("C.dat", C)
```

3. Program Structure

- readMatrixFromFile(filename, rows, cols) — reads a matrix, auto-detecting its shape from file content.
- multiplyMatrices(A, B, r1, c1, r2, c2) — checks compatibility and computes the product.
- displayMatrix(matrix, rows, cols) — prints a matrix to the console.
- writeMatrixToFile(filename, matrix, rows, cols) — writes a matrix to a file.
- main() — reads A and B, calls multiplyMatrices, displays and saves the result.

4. Input & Output File Format

A.dat and B.dat contain only the matrix values (no dimensions header) — one row per line, values separated by spaces, tabs, or commas. Rows must all have the same number of values. Example 2×3 matrix:

```
1 2 3
4 5 6
```

C.dat is written in the same plain format, with values fixed to 4 decimal places:

```
58.0000 64.0000
139.0000 154.0000
```

5. Test Cases

Test Case 1 — Standard rectangular multiplication

```
A.dat: 1 2 3 / 4 5 6
B.dat: 7 8 / 9 10 / 11 12
C.dat: 58.0000 64.0000 / 139.0000 154.0000
Result: Pass
```

Test Case 2 — Incompatible dimensions

```
A.dat: 1 2 / 3 4
B.dat: 1 2 / 3 4 / 5 6
C.dat: (not written – multiplication rejected)
Result: Pass — correctly rejected
```

Test Case 3 — Comma-separated values

```
A.dat: 1,2,3 / 4,5,6
B.dat: 7,8 / 9,10 / 11,12
C.dat: 58.0000 64.0000 / 139.0000 154.0000
Result: Pass
```

Test Case 4 — Decimal (non-integer) values

```
A.dat: 1.5 2.25 / 3.1 4.0
B.dat: 0.5 1.0 / 2.0 3.5
C.dat: 5.2500 9.3750 / 9.5500 17.1000
Result: Pass
```

Test Case 5 — Malformed file — inconsistent row lengths

```
A.dat: 1 2 3 / 4 5 (row 2 has only 2 values)
B.dat: 1 2 / 3 4
C.dat: Error: A.dat has an inconsistent number of values on line 2 (expected
3, found 2).
Result: Pass — correctly detected
```

6. Conclusion

The program correctly implements the required matrix-reading, compatibility-check, multiplication, and file-writing algorithm. Beyond the minimum requirements, it also handles:

- Auto-detects matrix dimensions from file content instead of requiring a fixed header format.

- Accepts values separated by spaces, tabs, or commas interchangeably.
- Ignores blank or trailing blank lines at the end of a file.
- Detects and reports malformed input — rows with an inconsistent number of values — with a clear error message and line number.
- Uses a configurable fixed decimal precision (OUTPUT_PRECISION) applied consistently to console output and C.dat.